

Technology Architecture Documentation

Integration between the Sync2Books Trader Invoicing System (TIS) and the KRA electronic Tax Invoice Management System (eTIMS) via OSCU

Prepared for	Kenya Revenue Authority — eTIMS Third-Party Integrator Certification / Go-Live
Product	Sync2Books — Trader Invoicing System (TIS)
Integration path	OSCU (Online Sales Control Unit), KRA-hosted
Document version	2.0
Date	2026-09-20
Supersedes	v1.0 (sync2books-TIS-Technology-Architecture.pdf)
Status	Describes the system as built and operating against the KRA sandbox. Items not yet built are confined to §9 and are explicitly labelled "planned".

Statement of accuracy. Every capability described in §1–§8 in the present tense corresponds to code running in the Sync2Books platform today and, where stated, to submissions verified against the KRA sandbox. Capabilities that are designed but not yet implemented appear only in §9 (Planned enhancements). Where a statement depends on a deployment-time configuration value rather than on code, that dependency is stated explicitly. v2.0 corrects four statements in v1.0 that described intent rather than implementation; the corrections are listed in §10.2.

Scope of disclosure. This document describes the fiscalisation path and the interface a taxpayer or their developer integrates against. It deliberately does not enumerate Sync2Books' internal service topology, internal routes or administrative interfaces: those are not part of the integration contract, and publishing them would widen the attack surface of a system that holds taxpayers' KRA device credentials without telling a reviewer anything about how fiscal data reaches KRA. Every control that bears on the integrity of that path is stated in §6 in terms of what it guarantees.

1. Purpose and scope

1.1 Purpose

This document describes the technology architecture through which Sync2Books operates as a **Trader Invoicing System (TIS)** and transmits fiscal data to the **KRA electronic Tax Invoice Management System (eTIMS)** over the **OSCU** path, on behalf of onboarded Kenyan taxpayers.

It is submitted in support of Sync2Books' application for eTIMS OSCU third-party integrator go-live, alongside the eTIMS Bio Data Form, Tax Compliance Certificate and proof of qualified technical staff.

1.2 What Sync2Books is

Sync2Books is a multi-tenant, cloud-hosted invoicing and tax-compliance platform. A taxpayer uses it in one of three ways, all of which converge on the same fiscalisation path:

1. **Through the Sync2Books compliance dashboard** — the taxpayer's finance staff create items, record sales and credit notes, and print receipts directly in a web application.
2. **Through their existing accounting platform** — Sync2Books reads invoices, credit notes, items, stock and purchases out of QuickBooks Online, Odoo, Microsoft Dynamics 365 Business Central or Xero, fiscalises them, and writes the KRA-signed receipt back onto the source document. The taxpayer does not change how they invoice.
3. **Through the Sync2Books Compliance API** — the taxpayer's own software, or a developer working for them, submits sales programmatically using an API key issued to that taxpayer from the compliance dashboard.

In all three cases the **same compliance service** performs the OSCU call, applies the same validation rules, and produces the same TIS-conformant receipt.

1.3 Scope

In scope: the fiscalisation path itself — the credential model, the OSCU call sequence, the integration surface a taxpayer or their developer works with, the printed/PDF receipt, security controls, environment separation, and error handling.

Out of scope: Sync2Books' internal service topology and any interface that is not part of the integration contract; the internal mechanics of each third-party accounting platform's own API; the taxpayer's commercial processes; billing and subscription handling.

1.4 Positioning — TIS, OSCU and eTIMS

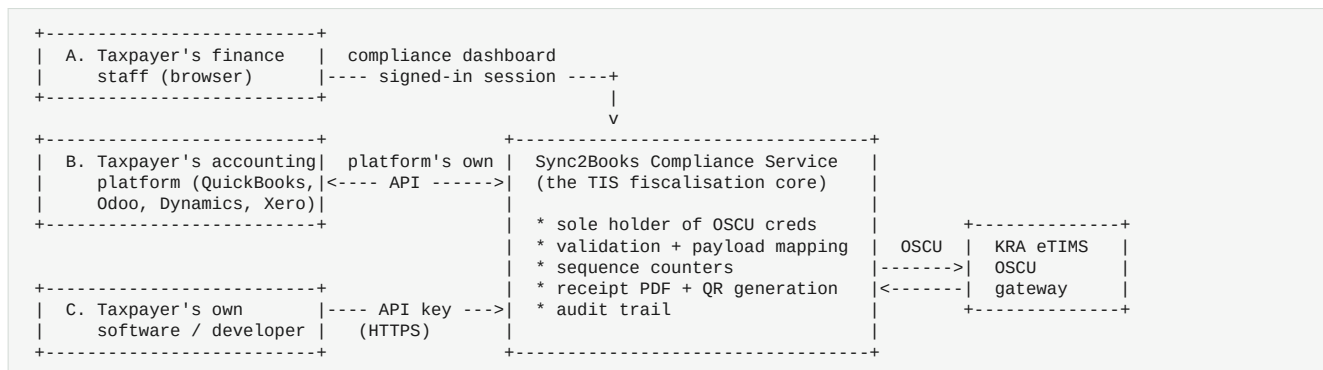
Role	Filled by
TIS (Trader Invoicing System) — the business software that issues the invoice	Sync2Books
OSCU — the KRA-hosted control unit that signs and numbers the transaction	KRA (etims-osc constuct gateway)
eTIMS — KRA's tax invoice management system of record	KRA

Sync2Books does **not** use VSCU. No signing hardware, control-unit appliance or client-side JAR is distributed to taxpayers; every OSCU call is made server-side from Sync2Books' own infrastructure. This is what makes a single centrally-operated platform able to serve many small and medium taxpayers.

2. System context

2.1 How work reaches KRA

A taxpayer's fiscal data can originate in three ways. All three converge on one component, and only that component speaks to KRA.



Source	How it is authorised	What it submits
A — Dashboard user	A signed-in session on the compliance dashboard, bound to the taxpayer's organisation	Items, sales, credit notes, stock movements and purchases entered by finance staff
B — Accounting platform	The taxpayer authorises Sync2Books in their own accounting platform, using that platform's consent flow	Invoices, credit notes, items, stock and purchases read out of the taxpayer's books
C — Developer integration	An API key the taxpayer issues from the compliance dashboard, scoped to that taxpayer's own businesses	Whatever the taxpayer's own software issues — typically sales and credit notes, with catalogue and stock kept in step

The single KRA path. No source reaches KRA directly. The accounting-platform connectors do not call KRA. A developer's API key does not reach KRA. The dashboard does not call KRA. Only the compliance service holds OSCU credentials and only it opens a connection to the KRA gateway. This is an architectural property, not a convention: the

credentials (cmckey, dvcId, sdcId) exist in exactly one database table, in one service, and the OSCU HTTP client lives in that same service.

2.2 The integration surface a developer sees

Path C is the one an external integrator works with, so it is worth stating concretely. A developer never handles an OSCU credential and never chooses a KRA endpoint.

1. **The taxpayer registers their business** in the compliance dashboard: KRA PIN, branch (bhfid), trade address and device serial. Sync2Books initialises the OSCU device for that branch (§4.1).
 2. **The taxpayer issues an API key** from the compliance dashboard, for their own developer. The key is scoped to that taxpayer's businesses and to one environment — a sandbox key can address only sandbox businesses, a live key only production ones — so a key can never act for another taxpayer or cross an environment boundary. *(Self-service issuance and rotation in the dashboard is being delivered; see §9.2. Today a key is issued to the taxpayer by Sync2Books on request, with the same scoping.)*
 3. **The developer syncs reference data** — KRA code lists and item classifications (§4.2) — and registers the taxpayer's items against them (§4.4). These are the values every later submission is validated against, so they are fetched from KRA rather than hard-coded by the integrator.
 4. **The developer submits sales and credit notes** (§4.6) and keeps stock in step (§4.5). Each submission is validated, mapped to the OSCU payload, given its KRA-owned sequence number and submitted; the KRA response is stored against the document.
 5. **The developer retrieves the receipt** (§4.8) — a TIS-conformant PDF with the verification QR — and presents or prints it. Submission status, KRA error detail and allocated receipt numbers are readable over the same API.
- The operations in §4 are the full surface. Everything a developer needs — reference data, catalogue, stock, sales, credit notes, purchases, lookups and receipts — is exposed there; anything else in the platform is internal and is not part of the integration contract.

2.3 The fiscalisation core

Concern	Where it lives
Taxpayer, branch and eTIMS-connection provisioning; device initialisation	Compliance service
OSCU credential custody	Compliance service — exclusively
Validation rules, OSCU payload construction, sequence counters, submission, KRA response handling	Compliance service
Receipt PDF and QR generation, reprints and COPY receipts	Compliance service
Regulatory audit trail	Compliance service
Reading the taxpayer's books from their accounting platform, and writing the signed receipt back onto the source invoice	Sync2Books accounting-platform sync (never touches KRA or OSCU credentials)

3. Component architecture

3.1 Credential custody — one row per branch

KRA issues OSCU operating credentials **per branch** (per bhfid of a taxpayer's PIN). Sync2Books models this literally.

```

compliance_tenants          one taxpayer (KRA PIN holder)
  id, sync2booksCompanyId, displayName, organizationId,
  receiptHeaderMessage, receiptFooterMessage
    | 1..n
    v
compliance_branches        one KRA branch office
  id, tenantId, sync2booksBranchId, displayName,
  kraBhfId,                <- KRA's own branch office code ("00", "02", ...)
  tradeAddressLine1, tradeCity <- printed on the receipt header
    | 1..1
    v
compliance_etims_connections the OSCU device/credential record for that branch
  id, complianceBranchId, sync2booksConnectionId,
  kraPin,                <- the branch's KRA PIN
  dvcSrlNo,              <- device serial sent in the initialize request
  deviceId,              <- OSCU dvcId returned by initialize
  cmcKey,                <- communication key returned by initialize
  sdcId,                 <- SCU ID (printed as "CU ID" on the receipt)
  mrcNo,
  environment,           <- SANDBOX | PRODUCTION
  status, lastCodeSyncAt

```

Properties that follow from this shape:

- **No pooled or shared credential across taxpayers in production.** Each branch's `cmcKey` and `dvcId` are the ones KRA issued to that branch's own device.
- **bhfId is never improvised.** Every OSCU request header takes `bhfId` from `compliance_branches.kraBhfId` — KRA's own code — and never from a Sync2Books-internal branch identifier. The two are deliberately different columns with different names so they cannot be transposed.
- **A branch is independently fiscalisable.** A taxpayer with several branches has several `compliance_etims_connections` rows; a sale is attributed to the connection of the branch it was issued at.
- **The environment is a property of the connection row**, so a sandbox-provisioned branch cannot address the production OSCU host (see §7).

Sandbox exception, stated for completeness. In the KRA **sandbox only**, test businesses may be provisioned against one already-initialised shared sandbox device (driven by `ETIMS_SANDBOX_SHARED_*` environment variables) rather than each calling `initialize` again. This exists because repeated `initialize` calls against one sandbox device serial have been observed to corrupt that device's record at KRA. It applies to sandbox test businesses only and has no production equivalent.

3.2 Modules inside the compliance service

Module	Responsibility	Principal OSCU operations
compliance-organization	Taxpayer, branch and eTIMS-connection provisioning; device initialisation	<code>initialize</code>
catalog	Item catalogue, item registration, KRA code lists and item classifications	<code>saveItem</code> , <code>selectCodeList</code> , <code>selectItemClass</code> / <code>selectItemClsList</code> , <code>itemInfo</code>
inventory	Per-branch stock ledger, stock in/out and stock-master reporting	<code>insertStockIO</code> , <code>saveStockMaster</code> , <code>selectStockMoveList</code>
sales	Compliance document lifecycle, validation rules, submission, receipt PDF + QR	<code>sendSalesTransaction</code> / <code>saveTrnsSalesOsd</code> , <code>selectInvoiceDetail</code> , <code>selectSalesTransactions</code>
dashboard-purchases	Inbound purchase transactions and confirmation	<code>getPurchaseTransactionInfo</code> , <code>sendPurchaseTransactionInfo</code>

Module	Responsibility	Principal OSCU operations
regulatory/oscu	Transport: OSCU HTTP client, endpoint path resolution, payload builders, envelope/error normalisation, gateway token handling, operation audit log	all of the above

The regulatory/oscu module is deliberately transport-only. It transforms a validated internal document into an OSCU payload and normalises the response; it holds no business rules. Business rules live in the domain modules, so a future regulator or a second regime can be added without rewriting the domain.

3.3 Layering inside a module

Every domain module uses the same four layers, which is what keeps the KRA-facing mapping in one place:

domain/ rules/ application/ infrastructure/ presentation/	entities, enums, state machine, invariants, idempotency key structural . tax . classification . PIN rule engines use cases (create -> validate -> prepare -> submit), ports OSCU payload builders, OSCU adapter, TypeORM persistence HTTP controllers + DTOs
---	--

3.4 Accounting-platform connectors are optional and upstream

The ERP connectors (QuickBooks Online, Odoo, Microsoft Dynamics 365 Business Central, Xero) live entirely in the Sync2Books accounting-platform sync, upstream of fiscalisation. They are an **optional data source**, not part of the fiscalisation path:

- A taxpayer who uses the Sync2Books dashboard directly has no ERP connection at all and fiscalises normally.
- A connector never holds an OSCU credential and never calls KRA.
- The only thing a connector does after fiscalisation is receive the signed receipt back and attach it to the originating invoice in the taxpayer's own books (§4.9).

Live-fire status of the connectors, stated plainly: **QuickBooks Online and Odoo have been exercised end-to-end against live accounts including receipt write-back.** Dynamics 365 Business Central and Xero are implemented and unit-tested but have not yet been verified against a live tenant. None of this affects the KRA-facing path, which is identical regardless of source.

4. Integration sequence

Sync2Books implements the OSCU operations in the dependency order KRA's step-by-step guide requires. The table below maps each stage to the module that owns it, the Sync2Books endpoint that drives it, and the OSCU operation it produces. Endpoint paths are given relative to the compliance service.

#	Stage	Sync2Books endpoint	OSCU operation
1	Device initialisation	POST /compliance-organization/branches/:branchId/etims-connection/initialize	initialize
2	Code list sync	POST /catalog/codes/sync	selectCodeList
3	Item classification sync	POST /catalog/item-classifications/sync	selectItemClass (integrator) / selectItemClsList (legacy)
4	Branch list	GET /oscu/branches	branchList
5	Notices	GET /oscu/notices	selectNoticeList
6	Taxpayer info	GET /oscu/taxpayer-info	selectTaxpayerInfo

#	Stage	Sync2Books endpoint	OSCU operation
7	Branch customer / user / insurance	POST /oscu/branches/customer, /user-account, /insurance	branchSendCustomerInfo, branchUserAccount, branchInsuranceInfo
8	Item registration	POST /catalog/items then POST /catalog/items/sync	saveItem
9	Item lookup / composition	GET /oscu/items/info, POST /oscu/items/composition	itemInfo, saveItemComposition
10	Imported items	GET /oscu/imported-items, POST /oscu/imported-items/convert	importedItemInfo, updateImportItem
11	Stock in/out and stock master	PUT /api/stock/adjust, POST /api/stock/transfer	insertStockIO, saveStockMaster
12	Stock movement lookup	GET /oscu/stock/movements	selectStockMoveList
13	Sales transaction	POST /api/sales	sendSalesTransaction / saveTrnsSalesOsdC
14	Credit note	POST /api/sales/credit-notes/express	sendSalesTransaction with credit-note receipt type
15	Sales lookups	GET /oscu/sales/invoice-detail, GET /oscu/sales/transactions	selectInvoiceDetail, selectSalesTransactions
16	Purchases	GET /oscu/purchases, POST /oscu/purchases	getPurchaseTransactionInfo, sendPurchaseTransactionInfo
17	Customer list	GET /oscu/customers	selectCustomerList
18	Receipt generation and reprint	GET /api/sales/:id/receipt[?copy=true]	— (local, from the stored KRA response)

These 18 stages cover all 23 test cases on KRA's go-live test dashboard. A per-test-case mapping, including the exact payloads accepted, is maintained internally and was used to produce the evidence set in §5.4.

4.1 Device initialisation

The provisioning call supplies { `tin`, `bhfId`, `dvcSrlNo` } to OSCU initialize. The response's `data.info` is parsed and persisted onto the branch's connection row: `cmckey`, `dvcId` (stored as `deviceId`), `sdCId` and `mrcNo`. Initialisation is refused before the branch has a `kraBhfId` and a device serial, and it is performed **once per device**. Until it has succeeded the connection is not usable for any other operation, because every subsequent OSCU request carries the `cmckey` in its headers.

4.2 Reference data: code lists and item classifications

KRA's code list (tax types, payment types, quantity and packaging units, receipt types) and the item classification list are pulled into local tables (`oscu_codes`, `oscu_code_classes`, `oscu_item_classifications`). Each list keeps its own `lastReqDt` watermark in `oscu_sync_state`, so an incremental sync fetches only what has changed; a full re-pull is available on demand.

Both lists are environment-wide rather than taxpayer-specific, so one active connection per environment is enough to authenticate the pull. A **scheduled daily job re-pulls both lists at 02:00** so that a newly provisioned taxpayer never faces an empty classification list and so that KRA-side code changes propagate without manual action.

4.3 Branch information

Branch list, notices and taxpayer information are read from OSCU and surfaced in the dashboard. Branch customer records, branch user accounts and branch insurance records are written to OSCU through `/oscu/branches/*`. These are pass-through operations: they carry no Sync2Books-side business rules, and each call's exact request and response is written to `oscu_operation_logs` (§8.5).

4.4 Item registration

An item is registered with `saveItem` before it may appear on any invoice line. The `itemCd` is constructed to KRA's format rather than being an opaque local identifier:

```
itemCd = orgnNatCd(2) + itemTyCd(1) + pkgUnitCd(2) + qtyUnitCd(2) + seq(7)
e.g.   KE           2           NT           BA           0000012
```

Three consequences are handled explicitly in the code:

- **seq is a real, strictly-incrementing, per-PIN counter starting at 1**, held in `oscu_sync_state` under `item_cd_seq:<kraPin>:<environment>`. It is never randomised and never reused. A permanent (non-retryable) rejection releases the reserved value so the counter cannot run ahead of KRA's.
- **The embedded `pkgUnitCd/qtyUnitCd` slices must match the flat fields in the same request**. Some legitimate KRA unit codes are not exactly two characters; where a taxpayer's real code does not fit the fixed-width slot, a valid two-character KRA code is substituted — and the identical substituted value is placed in the flat field, because KRA cross-checks the two.
- **`itemCd` encodes `itemTyCd`**. If an item is corrected from Goods to Service (or the reverse), the old `itemCd` no longer describes it. This is detected up front and a correct code is allocated in the same sync pass, rather than leaving documents pointing at a code KRA has no record of.

Registration status (PENDING | REGISTERED | FAILED), the assigned `itemCd`, the resolved classification code and the method by which it was resolved are all persisted on the catalogue row.

4.5 Stock

Stock is a per-branch ledger. A stock adjustment or transfer produces `insertStockIO` (the stock movement) and, where enabled, `saveStockMaster` (the resulting on-hand quantity). Two KRA-side rules are honoured:

- **All amounts in stock and sales payloads are tax-inclusive**. `taxblAmt` is derived by dividing the tax-inclusive total by the rate factor and `taxAmt` is the remainder; tax is never added on top.
- **`sarNo` is a strictly-incrementing per-PIN counter** held under `stock_sar_no:<kraPin>:<environment>`.

Its allocation runs inside a transaction with a row lock, so two concurrent stock calls serialise rather than both reading the same value. A rejected call releases the number.

`saveStockMaster`'s `rsdQty` is validated by KRA against the `insertStockIO` ledger it has accumulated, so the two are kept consistent and a mismatch is detected and repaired rather than retried blindly (§8.3).

4.6 Sales and credit notes

A sale follows an explicit state machine (§8.1). On `POST /api/sales`, the compliance service:

1. Creates the compliance document, deriving the idempotency key `merchantId:sourceDocumentId:documentType`. A replay of the same key returns the existing document rather than creating a second one.
2. Applies inventory movements for stocked goods.
3. **Validates** against four rule engines — structural (line count, totals reconcile), tax (rate bands), classification (valid classification code, unit present, goods vs service), and PIN (B2B vs B2C, malformed PIN rejected). A document that fails validation is never submitted.
4. **Prepares** the OSCU payload: refreshes each line's `itemCd`, `taxTyCd`, `pkgUnitCd/qtyUnitCd` snapshot so a frozen snapshot cannot be replayed after the item has changed; allocates `invNo` from `invoice_seq:<kraPin>:<environment>`.

5. **Submits** synchronously to OSCU and interprets the response (§8.2).

6. On acceptance, persists `curRcptNo`, `totRcptNo`, `rcptSign`, `intrlData`, `sdcdatetime`, the receipt label and the assigned `invcNo`, and records a `compliance_events` row holding both the payload and the response.

A **credit note** uses the same pipeline with the credit-note receipt type; quantities and tax amounts are normalised, the original sale is referenced, and the receipt carries the original CU invoice number (§5.2).

Goods cannot be sold below recorded stock; services are exempt from that check, which is why the goods/service distinction (`itemTyCd`) is carried on every item and line.

4.7 Purchases

Supplier transactions are read with `getPurchaseTransactionInfo`, matched to local supplier and item records, and confirmed with `sendPurchaseTransactionInfo`. Purchases carry their **own** `invcNo` counter, separate from sales, under `purchase_confirm_seq:<kraPin>:<environment>`.

4.8 Receipt generation

Once KRA accepts a sale, the receipt is rendered locally from the stored KRA response — no second call to KRA is needed, and a reprint therefore always reproduces exactly what KRA signed. The renderer produces a PDF with an embedded QR code pointing at KRA's own receipt-verification portal (§5.4).

4.9 Receipt write-back to the taxpayer's accounting platform

For taxpayers on the accounting-platform path, an accepted submission raises a durable work item that attaches the receipt PDF to the originating invoice in QuickBooks, Odoo, Dynamics or Xero. Attachment is guarded against duplication in three independent ways: the work item is keyed to the compliance document id; an existing non-failed item for the same document blocks a second enqueue; and the handler re-checks the accounting platform before pushing. A failed attachment is re-drivable. This step is downstream of KRA and cannot alter the fiscal record.

This stage is entirely downstream of KRA acceptance. If it fails, the fiscal record at KRA is unaffected.

5. Receipt and TIS conformance

5.1 Normal invoice — TIS specification page 8

The PDF receipt renders the fields the TIS for OSCU/VSCU Technical Specifications v2.0 sample requires, in the sample's order:

Region	Fields rendered
Header	KRA logo position, trade name, shop address and city, supplier PIN, document title (TAX INVOICE), QR code
Commercial message	Merchant-configurable header message
Parties	Invoice number and date; buyer name, buyer PIN (printed only when present — a blank buyer PIN is a legitimate walk-in sale and is never invented), buyer telephone; supplier name, branch and device
Item lines	Description, goods/service marker, unit price, quantity, and the line total suffixes with its tax designation (e.g. 6,720.00B) as the page 8 sample shows
Totals	SUB TOTAL, VAT, TOTAL (with currency)
Payment	Payment method and the amount paid
Item counter	ITEMS NUMBER (TIS §6.25)
Tax table	All five bands A/B/C/D/E printed unconditionally with rate, taxable amount and tax amount, defaulting to zero when unused

Region	Fields rendered
SCU INFORMATION (§6.23)	SCU date and time taken from the OSCU response's own <code>sdcdatetime</code> (not the document date); CU ID from <code>sdcdid</code> ; CU Invoice No. as <code>{sdcdid}/{curRcptNo}</code> ; receipt counter <code>curRcptNo/totRcptNo</code> with the receipt label; Internal Data and Receipt Signature, both dashed every four characters per §6.23.6–§6.23.7
TIS INFORMATION	TIS receipt number, date and time
Footer	Merchant-configurable footer message

The totals block is derived from the same tax-inclusive band split that was sent to KRA, so the printed receipt cannot disagree with the submitted payload.

5.2 Credit note — TIS specification page 10

The credit-note layout additionally renders:

- ORIGINAL CU INVOICE NO.# — the **CU/SCU receipt number of the original sale**, derived from the original compliance document, not the trader's own invoice number.
- The verbatim statement CREDIT NOTE IS APPROVED ONLY FOR ORIGINAL SALES RECEIPT.
- Per-band total lines in the credit-note form (TOTAL {band}-{rate}%), and TOTAL / TOTAL TAX in place of the sale's subtotal/VAT block.
- The NC receipt label on the CU invoice number.

5.3 COPY receipts — TIS §11

A reprint is requested with `?copy=true` on the receipt endpoint and is unambiguously marked as a copy in three ways simultaneously, per §11:

1. A large rotated COPY watermark behind the whole receipt.
2. A bold COPY designation below the receipt header and above the item section, rendered at more than twice the size of the amount text.
3. The receipt label is mapped to its copy form — NS → CS, NC → CC — per §4.3, so the label printed next to the CU invoice number identifies the document as a copy.

5.4 QR verification on KRA's portal — live-verified evidence

Each receipt's QR code encodes the KRA receipt-verification URL:

```
https://etims-sbx.kra.go.ke/common/link/etims/receipt/indexEtimsReceiptData?Data=<PIN><bhfId><rcptSign>
```

The Data payload is **positional fixed-width concatenation with no separators**: an 11-character KRA PIN, then the 2-character KRA branch code, then the 16-character receipt signature. The branch code is taken from `kraBhfId` — KRA's code — and the builder refuses to emit a URL at all if any segment is missing or the wrong width, on the principle that a malformed link is worse than none because it still renders as a scannable QR code on a customer's receipt. The host is selected from the connection's environment, so a sandbox submission only ever resolves on the sandbox portal.

Evidence — Gear Train Engineering Limited, 2026-09-18 (KRA sandbox). A 15-item catalogue (6 stocked goods, 9 non-stock services) across tax bands A, B, C and D, producing seven invoices and one credit note:

Document	CU invoice no.	Demonstrates
INV-260918-01	.../1	Single band B
INV-260918-02	.../2	Bands B + A (exempt) + C (zero-rated) on one invoice
INV-260918-03	.../3	Mixed goods and services, bands B and D, buyer PIN, cheque payment
INV-260918-04	.../4	Services only (non-stock), mobile-money payment

Document	CU invoice no.	Demonstrates
INV-260918-05	.../5	Zero-rated only
INV-260918-06	.../6	Exempt only, cash
INV-260918-07	.../7	Walk-in consumer, no buyer PIN, card
CN-260918-01	.../8	Partial credit of one goods line and one service line against .../3

Each document exists as both an original and a COPY. **Receipts .../3 and .../8 were scanned and verified on etims-sbx.kra.go.ke: the invoice number, client name and PIN returned by KRA's portal match the printed PDFs.** All 23 KRA go-live test-case endpoints returned resultCd 000, or 001 ("no search result", a valid empty result) for the stock-movement lookup; the raw responses are retained.

Known open template item. The official KRA logo asset is not bundled; a bordered placeholder holds its position on every receipt so the field is visibly reserved rather than silently omitted. Sync2Books requests the approved logo asset, or KRA's confirmation of the acceptable substitute, as part of go-live.

6. Security and credential custody

6.1 Credential custody

- KRA PIN, bhfId, device serial, cmcKey, dvcId and sdcId exist in **one table in one service** (compliance_etims_connections). They are never copied into another Sync2Books service, never sent to an accounting-platform connector, never returned over the API, and never returned to a browser.
- The OSCU HTTP client is the only code that reads cmcKey, and it places it in a request header to KRA and nowhere else.
- **Least privilege across taxpayers.** Each taxpayer's operations execute against that taxpayer's own branch connection, resolved from the request's tenant context. There is no pooled production OSCU credential.

6.2 Secrets management

Every secret — gateway client credentials, database credentials, internal service credentials, accounting platform OAuth client secrets — is supplied through the deployment environment. No credential, key or taxpayer PIN is held in source control, in test fixtures, or in configuration files committed to the repository. This is enforced as a standing engineering rule across all Sync2Books repositories.

6.3 Encryption in transit

All traffic to the KRA OSCU gateway, between Sync2Books services, to ERP platforms and to browsers uses TLS/HTTPS. The compliance dashboard is served from an explicit CORS allow-list rather than a wildcard, with credentials enabled only for those origins.

6.4 Authentication and authorisation

Caller	Control
Developer integration (API)	A per-taxpayer API key presented over TLS, scoped to that taxpayer's businesses and to a single environment. Keys are held as a hash, never in plaintext, and are revocable. Optional HMAC-SHA256 request signing is supported for callers that want message integrity as well as authentication.
Dashboard user	An authenticated session bound to the user's organisation; passwords are hashed with bcrypt.
Accounting platform	The platform's own OAuth2 / native credentials, obtained through that platform's consent flow and held encrypted by Sync2Books. Never a KRA credential.

Caller	Control
Internal service-to-service calls	Mutually authenticated over TLS with a deployment-issued secret compared in constant time, and bound to a single taxpayer per request (below). Webhook callbacks are verified with HMAC-SHA256 over the raw request bytes, so a re-serialised payload cannot silently pass verification.

Fail-closed authentication. Outside local development, a request on any route that can reach a taxpayer's KRA device credentials is refused when its credential is not configured, rather than admitted. An unset secret is treated as an open door, not as a convenience.

Every request is bound to one taxpayer. Authenticating a caller establishes *who is calling*; it never by itself establishes *which taxpayer they may act for*. Every route that names a taxpayer therefore re-checks that the named taxpayer is one the caller is entitled to act for, and rejects a mismatch with HTTP 403 — an API key cannot be pointed at another taxpayer's business, and a signed-in user cannot act outside their own organisation.

6.5 Encryption at rest

The compliance database is deployed on managed MySQL with storage-level encryption provided by the hosting platform. **Sync2Books does not currently apply application-level (field-level) encryption to the `cmckey`, device serial or ERP OAuth tokens.** v1.0 of this document stated that these fields were encrypted at the database layer; that statement was incorrect and is withdrawn. Field-level encryption for OSCU credentials is a committed near-term change (§9.1).

6.6 Rate limiting

Rate limiting today is a **per-caller request limit over a one-minute window, held in process memory**, and it does not emit `X-RateLimit-*` response headers. Stated plainly because the API is opening to external integrators: a shared-store limiter with published headers is a committed change before the developer surface is generally available (§9.2), and until then limits are not coordinated across instances.

v1.0 of this document listed **Redis** as the caching/rate-limiting technology. That was incorrect as a description of rate limiting and is withdrawn. Redis is used in exactly one place today: as an **optional** cache for the KRA gateway OAuth access token, selected when a Redis URL is configured and falling back to an in-process cache otherwise.

6.7 Multi-tenant isolation

Tenant isolation is enforced on every inbound surface before a request reaches the data layer:

- **Every request that names a taxpayer** is checked against the taxpayer the caller is entitled to act for, and a mismatch is rejected before any data is read (§6.4). This holds however the taxpayer is named — in the request body, the query string or the path.
- **Signed-in dashboard users** are resolved to their organisation from the session, and a business outside that organisation is refused.
- **Stock transfers**, which name two items and two branches and no taxpayer, are refused when the two items belong to different taxpayers, so stock cannot cross from one KRA device to another.
- A business with no owning organisation is refused rather than served to whoever asks for it first.

These checks are covered by unit tests that assert one organisation cannot read or write another's data. Making the taxpayer→organisation link mandatory at the schema level remains open (§9.1).

6.8 Auditability

Every fiscal operation leaves an immutable record; see §8.5.

7. Environments

7.1 Two environments, selected per branch

Environment	Purpose
SANDBOX	Development, integration testing and KRA go-live certification
PRODUCTION	Live submissions for certified branches

The environment is a column on the branch's eTIMS connection row, not a global deployment switch. The OSCU adapter resolves the base URL from that column on every single call, so:

- a branch provisioned in sandbox can never submit to the production OSCU host, and vice versa;
- one deployment can host sandbox and production taxpayers side by side without risk of cross-submission;
- the receipt QR URL host is selected from the same column, so a sandbox receipt verifies only on KRA's sandbox portal.

Reference data (code lists, item classifications) is synced independently per environment, so an environment with no connection yet does not block the other.

7.2 Gateway addressing

Sync2Books supports both OSCU deployment styles, selected by configuration:

Style	Base URL	Notes
Integrator (current, sandbox)	<code>https://sbx.kra.go.ke/etims-oscu/api/v1</code>	KRA's Apigee integrator gateway. Requires <code>tin</code> , <code>bhfId</code> , <code>cmckey</code> and the Apigee application id as request headers , plus an OAuth bearer token obtained by client-credentials and cached until shortly before expiry. Some resource paths differ from the specification's flat names (<code>sendSalesTransaction</code> , <code>insert/stockIO</code> , <code>save/stockMaster</code> , <code>selectItemClass</code> , <code>updateImportItem</code>); these are resolved from a single path table rather than being hard-coded at each call site.
Legacy / direct	<code>https://etims-api-sbx.kra.go.ke/etims-api</code> (sandbox), <code>https://etims-api.kra.go.ke/etims-api</code> (production)	Flat paths per OSCU Specification v2.0, with <code>tin/bhfId/cmckey</code> carried in the JSON body.

VERIFY (for Victor, before submission). The configured **production** default base URL is the legacy direct-KRA host (`https://etims-api.kra.go.ke/etims-api`), while the sandbox integrator default is the Apigee host. If go-live will run in integrator path style, the production integrator host must be confirmed with KRA and set explicitly via `ETIMS_OSCU_PROD_BASE_URL` — otherwise an integrator-mode production deployment would fall back to a legacy-style host. Please confirm the correct production integrator origin with KRA and state it here before this document is submitted.

7.3 Test data discipline

Certification and regression testing run exclusively against the KRA **sandbox**. No test exercises the production eTIMS environment. Test datasets use realistic business records — real-looking taxpayer names, items, addresses and amounts — rather than placeholder values, both because KRA reviewers open these records and because placeholder data does not exercise the same validation paths.

8. Reliability and error handling

8.1 Document state machine

A compliance document moves only along explicitly declared transitions; there are no implicit state jumps.

```

DRAFT --> VALIDATED --> READY_FOR_SUBMISSION --> SUBMITTED --+--> ACCEPTED (terminal)
|
| +--> REJECTED --+--> RETRYING --> SUBMITTED
|
| +--> FAILED
+--> CANCELLED (terminal)
+--> FAILED -----> RETRYING --> SUBMITTED

```

Enforced invariants:

- A document cannot reach ACCEPTED without having been SUBMITTED.
- Lines are frozen once the document is VALIDATED.
- An accepted document is immutable; a correction is a credit note, never an edit.
- Submission attempts are counted on the document.
- Every KRA response is stored (§8.5).

8.2 Interpreting a KRA response correctly

Three response behaviours are handled explicitly, because each one has been observed to mask a real rejection:

- **The gateway can wrap a genuine business rejection inside an outer HTTP 200** with a null response body and the real failure in the envelope's `responseHeader.responseCode / debugMessage`. Sync2Books inspects the envelope rather than trusting the HTTP status.

- **The integrator gateway nests the OSCU payload** inside `responseBody`, while legacy responses are flat.

Both are unwrapped at a single choke point so that every caller sees one consistent shape.

- `resultCd 001 ("no search result")` is a valid pass for a lookup, not a failure.

Every OSCU request and response passes through one instrumented function, so a submission always leaves a log record of whether it reached KRA and what KRA said — at log level, not debug, because production deployments suppress debug output.

8.3 Sequence counter self-healing

Four counters are local mirrors of counters KRA owns: the `itemCd` sequence, `sarNo`, the sales `invcNo`, and the purchases `invcNo`. All four can drift — typically because a taxpayer's PIN is also used by another system that consumes values Sync2Books never sees. A drifted counter can never converge by incrementing further, so each is repaired by **overwriting the local value with KRA's**:

Counter	Repair
<code>sarNo (insertStockIO)</code>	KRA names the expected value in the rejection ("Invalid sarNo: Expected: N but found: M"). It is parsed and the call is retried once with the correct value.
<code>invcNo (sales)</code>	KRA names the expected value ("Invc No: N is invalid, use the expected value: M"). Parsed and corrected inline.
<code>itemCd</code> sequence	KRA masks the expected value behind asterisks, so it cannot be parsed. Repair costs an <code>itemInfo</code> probe to discover the true maximum, then re-allocation.
<code>invcNo (purchases)</code>	Purchases carry an independent counter, repaired from the same message shape.

A related repair covers `saveStockMaster`'s `rsdQty`, which KRA validates against the `insertStockIO` ledger it has accumulated — a rejection in either of KRA's two wordings is recognised and the quantity reconciled.

Counters advance **only on an accepted KRA response**; a rejection releases the reserved value, so the local counter cannot run ahead of KRA's. Each parser is deliberately strict about message shape, so a rejection belonging to a neighbouring counter cannot be misclassified and correct the wrong number.

VERIFY (for Victor). The sales `invcNo`, `sarNo` and `itemCd` repairs have each been confirmed against live KRA sandbox rejections. The **purchases** `invcNo` repair is implemented on the assumption that the purchases endpoint rejects in the same message shape as sales, and has **not** yet been confirmed live. Either confirm it against the sandbox before submission, or leave this paragraph as written.

8.4 Retry and idempotency

Idempotency. Every compliance document carries a unique key `merchantId:sourceDocumentId:documentType`, enforced by a unique database index. Resubmitting the same source document returns the existing document rather than creating a second fiscal record. A retry of an already-allocated document reuses its allocated `invcNo` and its line snapshots, which is precisely what KRA expects next.

Retry model, described as it actually works. Sync2Books does **not** operate a background queue that re-attempts KRA submissions automatically until acknowledged; v1.0 of this document described such a queue and that description is withdrawn. What exists today is:

Mechanism	Behaviour
In-request correction and retry	A drift rejection (§8.3) is corrected and the call retried within the same request, bounded by a small retry budget.
Durable state, explicit re-drive	A failed document persists in REJECTED or FAILED with KRA's error text. It is re-drivable at any time — individually or in bulk — from the dashboard or by API. The retry path walks a document forward through the real state machine (DRAFT → validate → prepare → submit, REJECTED/FAILED → RETRYING → submit); it never jumps states.
Durable queue for receipt write-back	The accounting-platform receipt attachment is a persisted work item, processed in the background and re-drivable if it fails. This is downstream of KRA and does not affect the fiscal record.
Scheduled reference-data sync	The daily 02:00 code-list and classification pull is the one automatic scheduled job on the KRA path.

The distinction matters and is stated deliberately: **a submission that KRA rejects is never silently retried into acceptance.** It is preserved, attributed, surfaced with KRA's own error text, and re-driven under operator control. Automatic scheduled retry with exponential backoff is a planned enhancement (§9.2), and it is listed there because it is not built.

Non-retryable failures are not retried. Validation failures, missing mappings and business-rule violations are terminal and surfaced for correction; only transport-level failures and recognised drift rejections are re-attempted.

8.5 Audit trail

Two complementary records, both append-only:

Table	Contents
<code>compliance_events</code>	One row per lifecycle event on a document — created, validated, submitted, accepted, rejected, retry attempted — each carrying the payload snapshot and the response snapshot . This is the regulatory record of what was sent and what KRA returned.
<code>oscu_operation_logs</code>	One row per raw OSCU pass-through call (branch customer/user/insurance, lookups, imported items, purchases): the operation name, merchant, branch, KRA <code>bhfId</code> , request body, success flag, <code>resultCd</code> , <code>resultMsg</code> , the full raw response, and any error.

Together these allow any historical submission to be reconstructed exactly, which is how the go-live evidence set in §5.4 was produced. Accepted documents additionally retain `curRcptNo`, `totRcptNo`, `rcptSign`, `intrlData`, `sdcdatetime`, the receipt label and the allocated `invcNo`, which is what makes a reprint reproduce the signed original rather than re-derive it.

8.6 Status visibility

Branch connection status, per-document compliance status, KRA error detail, allocated receipt numbers and receipt availability are all surfaced to the taxpayer in the compliance dashboard, and to integrators through the API.

9. Planned enhancements

Everything in this section is **designed but not yet implemented**. It is listed so that KRA has a complete picture of the platform's direction, and it is separated from §1–§8 so that nothing here can be mistaken for a current capability.

9.1 Committed near-term (security and isolation)

Items 1–4 below were completed on 2026-09-20 and are described in the present tense in §6.4 and §6.7; they are listed here so the change is traceable against v1.0 of this document.

1. ~~Fail closed authentication~~ — done. A route that can reach a taxpayer's KRA device credentials refuses the request outside local development when its credential is not configured (§6.4).
2. ~~Bind every request to one taxpayer~~ — done. A named taxpayer that disagrees with the one the caller authenticated for is rejected with HTTP 403.
3. ~~Verify business ownership wherever a taxpayer is named~~ — done, alongside the existing session-derived organisation check.
4. ~~Assert both items in a stock transfer belong to one taxpayer~~ — done.
5. **Make the taxpayer→organisation link mandatory** for newly created taxpayers (still open).
6. **Field-level encryption at rest for OSCU credentials** (cmckey, device serial) and accounting-platform OAuth tokens (see §6.5) (still open).
7. **Derive the taxpayer from the session on every remaining surface** so a client-supplied identifier is never the only scoping input (still open).

9.2 Platform roadmap

Item	Description
Self-service API key issuance	The taxpayer will issue, rotate and revoke their own API keys from the compliance dashboard, without a Sync2Books operator in the loop (§2.2). The scoping and storage described in §6.4 — one taxpayer, one environment, hashed at rest, shown once — apply to keys issued today and continue to apply; what is being added is the self-service surface, per-key scopes, and a usage view.
Versioned /v1 API surface	A stable, versioned form of the operations in §4 — businesses, branches, items, stock, sales, credit notes, receipts and lookups — with a consistent error envelope and cursor pagination. The raw OSCU pass-through operations stay a certification tool and are not part of the developer surface.
Shared-store rate limiting with published headers	Per-key limits backed by a shared store, emitting X-RateLimit - * response headers (see §6.6).
Outbound webhooks	document.accepted, document.rejected, item.registered, stock.synced, receipt.ready and similar events, signed with HMAC-SHA256 over the raw body and timestamped against replay, with exponential backoff, a real scheduler, and a dead-letter view.
Automatic scheduled retry with backoff	A scheduled re-drive of transport-failed submissions with exponential backoff and a dead-letter state, replacing today's operator-driven re-drive (see §8.4).
Daily reconciliation	A scheduled comparison of locally recorded receipt numbers and stock quantities against KRA's own summaries, flagging mismatches for review.
Discount fields on the receipt	Carry and render discount rate, discount amount, TOTAL BEFORE DISCOUNT and TOTAL DISCOUNT AWARDED. The fields are currently submitted as zero because the platform does not yet capture line discounts.
KRA logo asset	Replace the reserved placeholder once the approved asset is supplied (see §5.4).

10. Glossary and change record

10.1 Glossary

Term	Meaning
TIS	Trader Invoicing System — the invoicing software that connects to eTIMS. Sync2Books is the TIS.

Term	Meaning
eTIMS	electronic Tax Invoice Management System — KRA's tax invoice system of record.
OSCU	Online Sales Control Unit — the KRA-hosted control unit that signs and numbers transactions. The path Sync2Books uses.
VSCU	Virtual Sales Control Unit — the alternative, taxpayer-hosted path. Not used by Sync2Books.
cmcKey	Communication key issued by KRA at device initialisation; authenticates every subsequent OSCU call for that branch.
dvcId / deviceId	OSCU device identifier returned by initialize.
dvcSrlNo	Device serial number sent in the initialize request. Distinct from dvcId.
sdId	SCU identifier returned by initialize (e.g. KRACU04...). Printed on the receipt as CU ID and forming the first half of the CU Invoice No.
bhfId	KRA branch office code ("00", "02", ...). Always KRA's own code, never a Sync2Books identifier.
itemCd	KRA item code: orgnNatCd(2) + itemTyCd(1) + pkgUnitCd(2) + qtyUnitCd(2) + seq(7).
itemTyCd	Item type code; 3 denotes a service. Embedded in itemCd.
invNo	Strictly incrementing per-PIN invoice sequence owned by KRA and mirrored locally. Sales and purchases have separate counters.
srNo	Strictly incrementing per-PIN stock-movement sequence.
curRcptNo / totRcptNo	Current receipt number for its type, and total receipt counter across all types; printed together as the receipt counter.
rcptSign	Receipt signature returned by OSCU; printed dashed every four characters and encoded into the verification QR code.
intrlData	Internal data returned by OSCU; printed dashed every four characters.
sdDateTime	The SCU's own date and time for the transaction; printed in the SCU information block.
Receipt label	TIS §4.3 two-letter designation — NS normal sale, NC normal credit note, CS/CC their copies, and so on.
merchantId	Sync2Books' identifier for a taxpayer (a business) within the platform. Not a KRA field.

10.2 What changed from version 1.0

#	v1.0 statement	v2.0 treatment	Why
1	"Caching / rate limiting: Redis "	Corrected in §6.6	Rate limiting today is a per-caller in-process counter, not a Redis-backed one. Redis is used only as an optional cache for the KRA gateway access token.
2	"Outbound eTIMS operations are written to a durable sync queue ... retried until KRA acknowledges it"	Corrected in §8.4; automatic scheduled retry moved to §9.2	KRA submission is synchronous. Retry is in-request drift correction plus durable state with operator-driven re-drive. A durable queue does exist, but for ERP receipt write-back, which is downstream of KRA.

#	v1.0 statement	v2.0 treatment	Why
3	"Encryption at rest. Sensitive connection fields (communication key, device serial) are encrypted at the database layer "	Corrected in §6.5; field-level encryption moved to §9.1	No application-level field encryption exists today. Storage-level encryption is provided by the hosting platform.
4	"Persistence: MySQL (TypeORM), with versioned SQL migrations "	Restated accurately	The compliance service's schema is derived from its entity definitions rather than from a versioned migrations directory.
5	Integration sequence described at component level	Rewritten in §4 with the concrete endpoint and OSCU operation for each of 18 stages	KRA asked how the integration takes place; naming the real endpoints and operations demonstrates it.
6	No receipt/TIS conformance section	Added as §5, with the live-verified Gear Train evidence set	Responds directly to the September template feedback on specification pages 8 and 10.
7	No statement of unbuilt work	Added as §9, clearly separated	So that no planned capability can be read as a current one.
8	Internal service topology described by name	Removed; §2 now describes the integration surface instead	The internal decomposition is not part of the integration contract, and naming internal services, routes and administrative interfaces in a published document widens the attack surface around taxpayers' KRA credentials without informing the review.

Sync2Books — Technology Architecture Documentation v2.0 — prepared for KRA eTIMS OSCU third-party integrator go-live.